

SISTEMAS OPERACIONAIS

SEMANA 1 - AULA 1

Gilberto Falco Netto

Tecnólogo em Análise e Desenvolvimento de Sistemas,
Faculdade Anhanguera - Jundiaí/SP

O que são Sistemas Operacionais?

- Software que gerencia hardware e software do computador.
- Exemplos: Windows, Linux, MacOS, Android.
- Funções principais: Gerenciamento de processos, memória, dispositivos e arquivos.

Importância dos Sistemas Operacionais

- Fornecem uma interface entre o usuário e o hardware.
- Otimizam a utilização dos recursos do sistema.
- Garantem a segurança e estabilidade do sistema.

Definição de Kernel

- Núcleo do sistema operacional.
- Controla e gerencia recursos do sistema.
- Facilita a comunicação entre hardware e software.

Função do Kernel

- Gerenciamento de processos.
- Gerenciamento de memória.
- Gerenciamento de dispositivos.
- Gerenciamento de sistemas de arquivos.

Batch OS

- Processa lotes de tarefas sem interação direta do usuário.
- Exemplo: Mainframes antigos.

Time-Sharing OS

- Suporte a múltiplos usuários simultaneamente.
- Usa técnicas de multitarefa para compartilhar tempo de CPU.
- Exemplo: Unix.

Distributed OS

- Gerencia múltiplos computadores interconectados.
- Transparência de recursos distribuídos.
- Exemplo: Plan 9.

Network OS

- Gerencia recursos em rede.
- Suporte a compartilhamento de arquivos e impressoras.
- Exemplo: Novell NetWare.

Real-Time OS

- Respostas em tempo real para eventos críticos.
- Usado em sistemas embarcados e industriais.
- Exemplo: VxWorks.

Kernel Monolítico

- Todas as funções essenciais integradas em um único bloco.
- Vantagens: alto desempenho, controle direto de hardware.
- Desvantagens: difícil de manter e depurar.
- Exemplo: Kernel Linux.

Microkernel

- Funções mínimas no núcleo; serviços executados no espaço do usuário.
- Vantagens: modularidade, facilidade de manutenção.
- Desvantagens: desempenho inferior devido à comunicação entre módulos.
- Exemplo: Kernel Minix.

Exokernel

- Núcleo mínimo e simples.
- Vantagens: eficiência, flexibilidade.
- Desvantagens: complexidade de desenvolvimento.
- Exemplo: Exokernel do MIT.

Nanokernel

- Núcleo extremamente pequeno, muitas vezes focado em uma única tarefa.
- Vantagens: alto desempenho, simplicidade.
- Desvantagens: limitações funcionais.

Gerenciamento de Processos

- Criação, escalonamento e terminação de processos.
- Comunicação e sincronização entre processos.
- Exemplo: Troca de contexto entre processos.

Gerenciamento de Memória

- Alocação e desalocação de memória.
- Paginação e segmentação para memória virtual.
- Exemplo: Tabela de Páginas.

Gerenciamento de Dispositivos

- Interação com dispositivos de entrada/saída.
- Drivers de dispositivos.
- Exemplo: Driver de impressão.

Gerenciamento de Sistemas de Arquivos

- Estrutura de diretórios e arquivos.
- Permissões de acesso.
- Exemplo: Sistema de arquivos NTFS.

Chamadas de Sistema

- Interface para serviços do sistema operacional.
- Exemplos: `fork()`, `exec()`, `read()`, `write()`.

Modos de Operação

- Modo Usuário: execução de aplicações.
- Modo Kernel: execução de operações privilegiadas.
- Troca de Modos: quando uma chamada de sistema é realizada.

Segurança no Kernel

- Controle de acesso aos recursos do sistema.
- Isolamento de processos para evitar interferências.
- Proteção de memória para evitar acessos não autorizados.

Kernel Space vs User Space

- Kernel Space: espaço de memória onde o kernel é executado.
- User Space: espaço de memória onde as aplicações do usuário são executadas.
- Comunicação entre os espaços: chamadas de sistema e interrupções.

Inicialização do Sistema Operacional

- Processo de boot: Inicialização do hardware e carregamento do kernel.
- Exemplo: BIOS/UEFI, carregador de boot (GRUB), inicialização do kernel.

Otimizações e Performance

- Técnicas de otimização: escalonamento eficiente, gerenciamento de memória avançado.
- Impacto na performance: melhora a velocidade e eficiência do sistema.

Atualizações de Kernel

- Importância: segurança, suporte a novo hardware, correção de bugs.
- Processo de atualização: substituição do kernel antigo, reinicialização do sistema.

Estudos de Caso: Linux

- História: Criado por Linus Torvalds em 1991.
- Características: código aberto, flexível, amplamente utilizado.

Estudos de Caso: Windows

- História: Desenvolvido pela Microsoft.
- Características: popular em desktops, compatível com uma ampla gama de software.

Estudos de Caso: MacOS

- História: Desenvolvido pela Apple Inc.
- Características: kernel XNU, design amigável, forte integração com hardware.

Kernel de Tempo Real

- Características: Respostas em tempo real, previsibilidade.
- Exemplos de uso: Sistemas industriais, automotivos, de saúde.

Kernel em Dispositivos Móveis

- Kernel Android: Baseado no kernel Linux, adaptado para dispositivos móveis.
- Kernel iOS: Baseado no kernel XNU, otimizado para dispositivos da Apple.

Kernel Modular

- Permite adicionar ou remover funcionalidades sem reiniciar o sistema.
- Exemplo: Módulos de kernel no Linux.

Kernel de Código Aberto

- Promove a colaboração e inovação.
- Exemplo: Kernel Linux, disponível para desenvolvedores e entusiastas.

Desafios no Desenvolvimento de Kernel

- Complexidade e necessidade de alta segurança.
- Compatibilidade com diversos hardwares.

Oportunidades para Inovações

- Desenvolvimento de novos métodos de gerenciamento de recursos.
- Melhorias na segurança e eficiência do kernel.

Conclusão

- Resumo dos pontos principais abordados.
- Perguntas e Respostas.